

CSE221: Algorithms

Comparison Based Sorting Algorithms

Prepared by: A B M Muntasir Rahman [MUNR]



Inspiring Excellence

Bubble Sort

- A simple sorting algorithm that works by comparing two adjacent elements and swaps them until they are in correct order.
- Just like air bubbles in waters rises up to the surface, at the end of each pass in Bubble Sort highest element moves to the end (while sorting in ascending order) or lowest element moves to the end (while sorting in descending order).

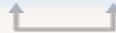
Index	0	1	2	3	4	5	6
Iteration							
1	-11	6	18	2	-34	32	68
		↕					
2	-11	6	18	2	-34	32	68
		↕					
3	-11	6	18	2	-34	32	68
			↕				
4	-11	6	2	18	-34	32	68
				↕			
	-11	6	2	-34	18	32	68

SW Test Academy

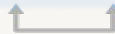
Bubble Sort (Simulation)

step = 0

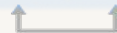
i = 0



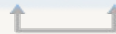
i = 1



i = 2



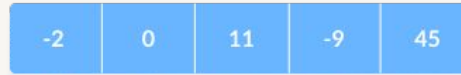
i = 3



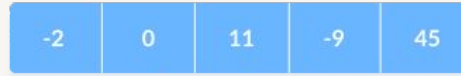
Bubble Sort (Simulation)

step = 1

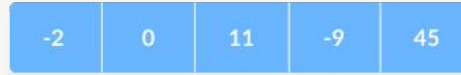
i = 0



i = 1



i = 2



Bubble Sort (Simulation)

step = 2

i = 0



i = 1



Bubble Sort (Pseudocode)

```
bubbleSort(array)
  for i <- 0 to sizeOfArray - 1
    for j <- 0 to sizeOfArray - 1 - i
      if leftElement > rightElement
        swap leftElement and rightElement
    end bubbleSort
```

Optimized Bubble Sort (Pseudocode)

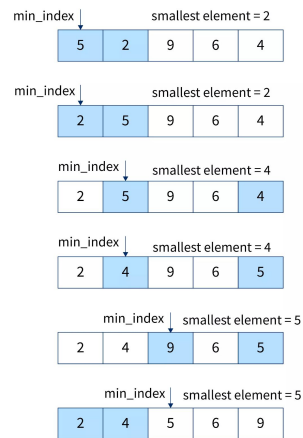
```
bubbleSort(array)
  for i <- 0 to sizeOfArray - 1
    swapped <- false
    for j <- 0 to sizeOfArray - 1 - i
      if leftElement > rightElement
        swap leftElement and rightElement
        swapped <- true
    if swapped equals false
      break
  end bubbleSort
```

Bubble Sort (Properties)

- Time Complexity:
 - Best Case: $O(n)$ when array is sorted already.
 - Average Case: $O(n^2)$ when array is in random order.
 - Worst Case: $O(n^2)$ when array is in reverse order.
- Space Complexity:
 - Best/Average/Worst case: $O(1)$.
- Stable: Yes.
- Inplace: Yes.
- Suitable for smaller arrays. Unsuitable for larger arrays and when lesser swapping is preferred.

Selection Sort

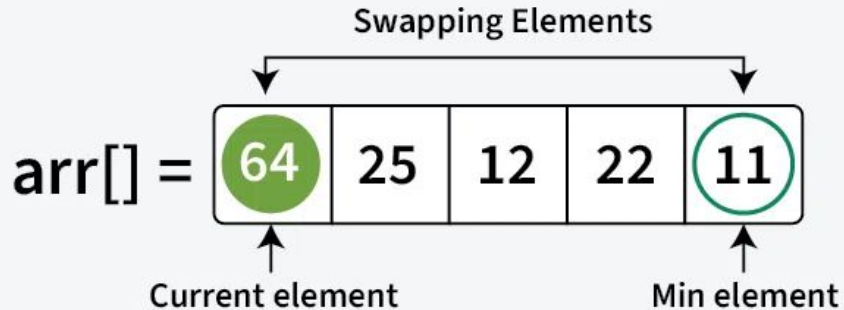
- A simple sorting algorithm that selects the lowest value in an array and moves it to the front of the array.
- The basic logic is to select the smallest element from the unsorted part and swap it with the min_index element.



Selection Sort (Simulation)

01
Step

Start from the first element at index 0, find the smallest element in the rest of the array which is unsorted, and swap (11) with current element(64).

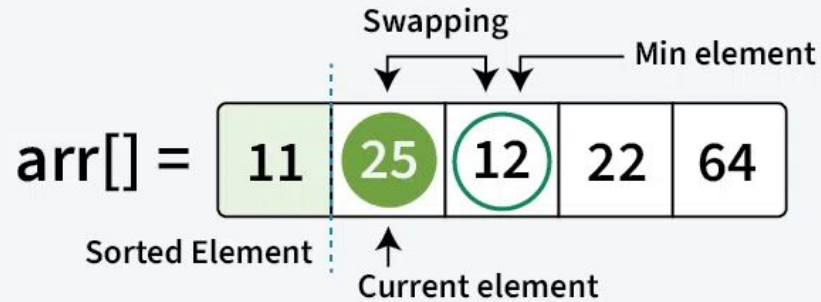


Selection Sort Algorithm

Selection Sort (Simulation)

02
Step

Move to the next element at index 1 (25). Find the smallest in unsorted subarray, and swap (12) with current element (25).

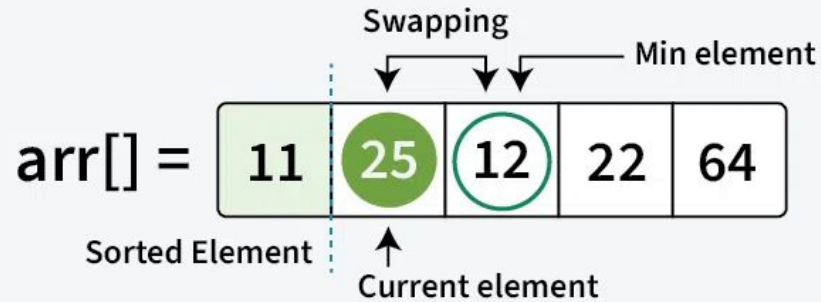


Selection Sort Algorithm

Selection Sort (Simulation)

02
Step

Move to the next element at index 1 (25). Find the smallest in unsorted subarray, and swap (12) with current element (25).

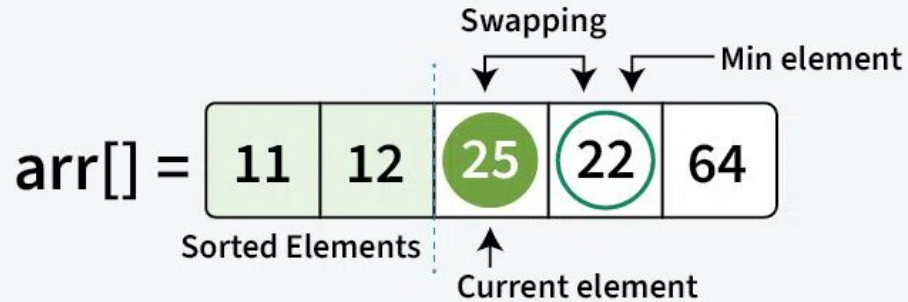


Selection Sort Algorithm

Selection Sort (Simulation)

03
Step

Move to element at index 2 (25). Find the minimum element from unsorted subarray, Swap (22) with current element (25).

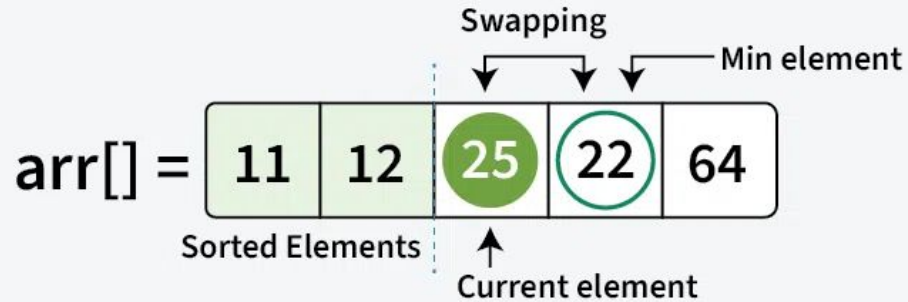


Selection Sort Algorithm

Selection Sort (Simulation)

03
Step

Move to element at index 2 (25). Find the minimum element from unsorted subarray, Swap (22) with current element (25).

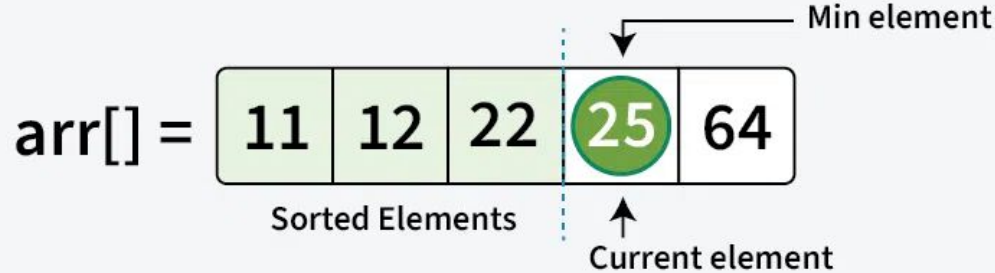


Selection Sort Algorithm

Selection Sort (Simulation)

04
Step

Move to element at index 3 (25), find the minimum from unsorted subarray and swap (25) with current element (25).

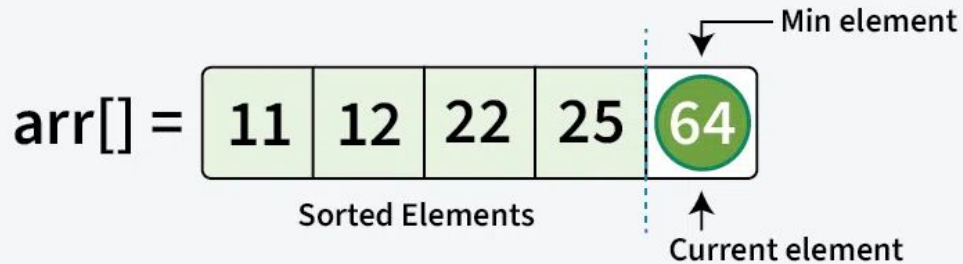


Selection Sort Algorithm

Selection Sort (Simulation)

05
Step

Move to element at index 4 (64), find the minimum from unsorted subarray and swap (64) with current element (64).



Selection Sort Algorithm

Selection Sort (Simulation)

06
Step

We get the sorted array at the end.

arr[] =

11	12	22	25	64
----	----	----	----	----

Sorted array

Selection Sort Algorithm

Selection Sort (Pseudocode)

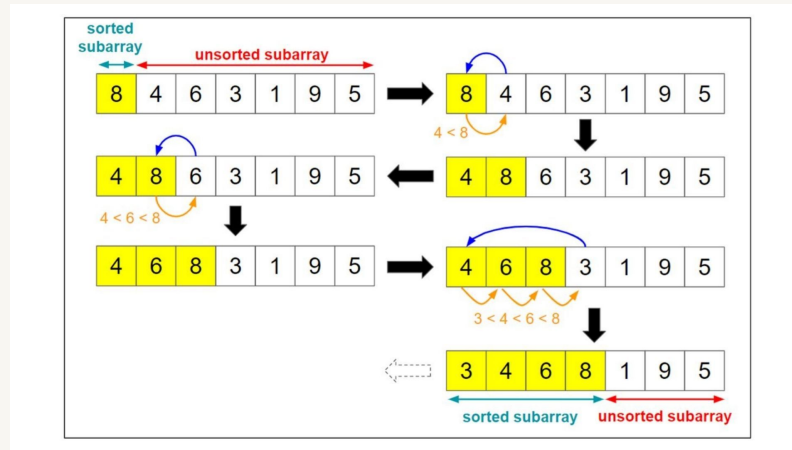
```
selectionSort(array, size)
  for i from 0 to size - 1 do
    set i as the index of the current minimum
    for j from i + 1 to size do
      if array[j] < array[current minimum]
        set j as the new current minimum index
    if current minimum is not i
      swap array[i] with array[current minimum]
  end selectionSort
```

Selection Sort (Properties)

- Time Complexity:
 - Best Case: $O(n^2)$ when array is sorted already.
 - Average Case: $O(n^2)$ when array is in random order.
 - Worst Case: $O(n^2)$ when array is in reverse order.
- Space Complexity:
 - Best/Average/Worst case: $O(1)$.
- Stable: No.
- Inplace: Yes.
- Suitable for smaller arrays and when lesser swapping is preferred. Unsuitable for larger arrays.

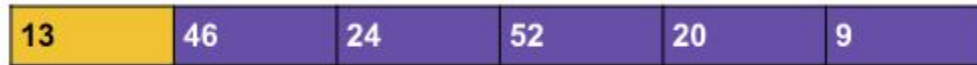
Insertion Sort

- A simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in sorted list by comparing.

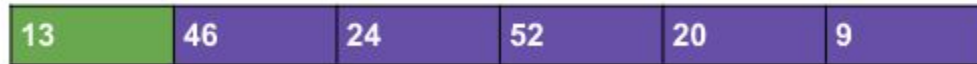


Insertion Sort (Simulation)

Selected element: 13 (index $i = 0$)



13 is in its correct position as no elements before it



Insertion Sort (Simulation)

Selected element: 46 (index $i = 1$)

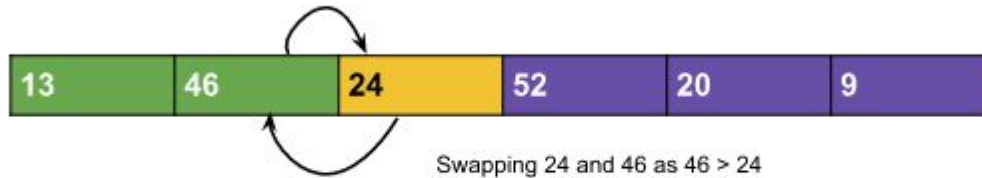


No swapping as $13 < 46$



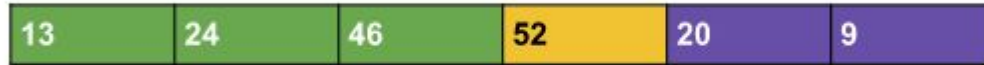
Insertion Sort (Simulation)

Selected element: 24 (index $i = 2$)

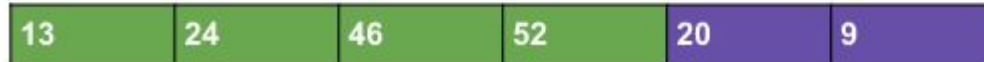


Insertion Sort (Simulation)

Selected element: 52 (index $i = 3$)



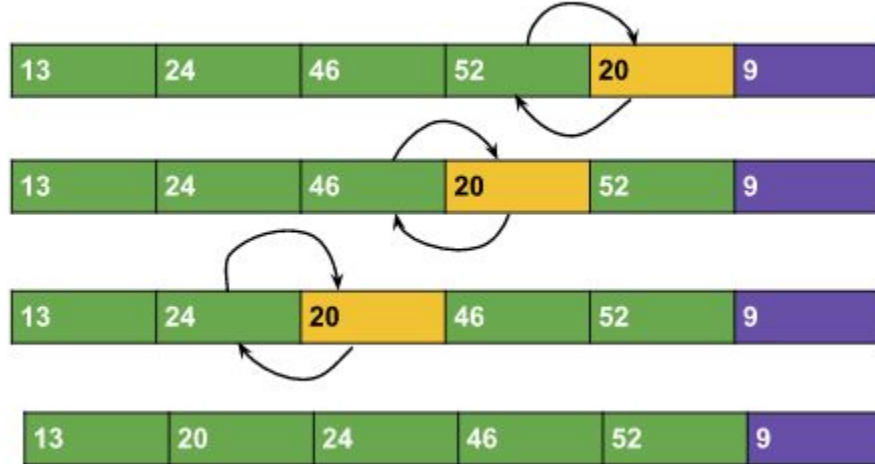
No swapping as $46 < 52$



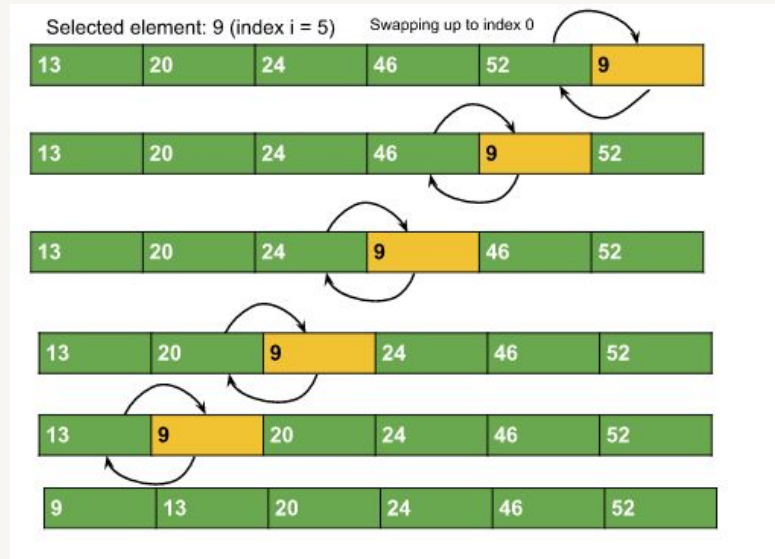
Insertion Sort (Simulation)

Selected element: 20 (index $i = 4$)

Swapping up to index 1



Insertion Sort (Simulation)



Insertion Sort (Pseudocode)

```
insertionSort(array)
  mark first element as sorted
  for each unsorted element X
    'extract' the element X
    for j <- lastSortedIndex down to 0
      if current element j > X
        move sorted element to the right by 1
    break loop and insert X here
  end insertionSort
```

Insertion Sort (Properties)

- Time Complexity:
 - Best Case: $O(n)$ when array is sorted already.
 - Average Case: $O(n^2)$ when array is in random order.
 - Worst Case: $O(n^2)$ when array is in reverse order.
- Space Complexity:
 - Best/Average/Worst case: $O(1)$.
- Stable: Yes.
- Inplace: Yes.
- Suitable for smaller arrays and when a few elements need to be sorted. Unsuitable for larger arrays.